

Technical Report: Bytecode Virtual Machine Implementation

Course Code: COD7001

Mayur Kamble 2025MCS2116

Nizaul Rahman 2025MCS2099

1 Architecture Overview

The system implements a stack-based virtual machine (VM) designed for deterministic execution of arithmetic, logic, and control flow instructions. Unlike register-based architectures, this VM performs operations directly on a primary data stack.

1.1 Memory Model

The VM utilizes a segmented memory architecture as per Lab 4 specifications:

- **Program Memory:** An integer array storing the loaded bytecode instructions and operands.
- **Data Stack:** A 256-slot array for intermediate calculations and operand processing.
- **Global Memory:** 1024 words of addressable storage, accessed via `STORE` and `LOAD` instructions.
- **Return Stack:** A dedicated 64-slot stack used exclusively for storing return addresses ($PC + 1$) to manage subroutine calls and function scopes.

2 Instruction Dispatch Strategy

The VM core follows a high-performance Fetch-Decode-Execute cycle:

- **Fetch:** The VM retrieves the opcode at the current Program Counter (PC) and increments the PC .
- **Decode/Execute:** A centralized `switch` statement acts as the dispatcher, branching to specific logic based on the opcode.
- **Operand Management:** Instructions with arguments (like `PUSH` or `JMP`) perform a secondary fetch from the program memory and advance the PC .

3 Subroutine and Call Frame Design

The VM supports modular programming through a robust Call/Return mechanism:

- **CALL Logic:** Upon a `CALL` instruction, the current PC is saved to the `return_stack`, and the PC is updated to the target subroutine address.
- **RET Logic:** The `RET` instruction pops the saved address from the `return_stack` back into the PC , allowing the program to resume immediately.
- **Isolation:** A separate return stack prevents function return addresses from being corrupted by data stack operations.

4 Performance & Benchmarks

Performance was evaluated using iterative, nested, and memory-intensive programs. All benchmarks were performed on a standard Linux environment.

Table 1: System Performance Evaluation

Program	Requirement Demonstrated	Time (Real)	Result
simple_fib.bin	Arithmetic & Expression Evaluation	0.002s	3
fib.bin	Non-trivial Iterative Computation	0.002s	13
nested.bin	Nested Function Calls	0.002s	35
mem_test.bin	Memory Load/Store & Looping	0.002s	55

5 System Robustness

- **Stack Safety:** The VM includes boundary checks for both the data stack and return stack.
- **Memory Integrity:** All allocated program memory is explicitly freed at execution end to ensure no memory leaks.
- **Instruction Validation:** The assembler (`asm.py`) performs error checking during translation.

6 Limitations and Enhancements

- **Limitations:** Static memory (1024) and stack (256) limits prevent handling extremely large datasets. Currently only supports 32-bit integer math.
- **Enhancements:** Future versions could include a standard library or JIT (Just-In-Time) compilation.